You said:

I'm working on a Python/Pygame project called Arm Wrestle Showdown for a software engineering lab.

The project is an arm-wrestling tug-of-war game. The player alternates between the Left and Right arrow keys to push, while the computer applies opposing force.

The project has this structure:

```
arm_wrestle/
├── game/
|   └── game_engine.py
├── main.py
└── README.md
```

The README identifies four tasks:

1. Fix the inverted arm push bug. Currently, alternating Left and Right arrow keys pushes the hands toward the computer's side instead of toward the player's winning threshold.
2. Implement a dynamic AI surge cycle, with periodic high-power surges followed by reduced resistance during cooldown.
3. Add a visual warning when the AI enters its power surge and an exhaustion indicator when player stamina drops too low.
4. Implement a counter-surge bonus. If the player pushes as the AI's surge ends and cooldown begins, temporarily double push strength and boost stamina recovery.

Other expected behavior:

* Rapidly alternating arrow keys drains player stamina.
* When stamina falls below 10, player input temporarily stops until stamina recovers.
* Reaching -100.0 means the player wins.
* Reaching +100.0 means the computer wins.
* Pressing R on the Game Over screen resets the stamina, arm position, and game states.

I want to use an iterative, AI-assisted debugging process. Do not immediately rewrite the entire project or invent code without examining the existing implementation.

First, ask me to provide the contents of main.py and game/game_engine.py. Then analyze the existing code, identify the likely cause of the inverted push bug, and explain the proposed fix before implementing it.

Keep the solution compatible with the existing project structure and preserve working functionality.

ChatGPT said:

Alright bro 😭🤝 We'll debug **Arm Wrestle Showdown** step by step, starting with your existing code. We won't rewrite the project or change working functionality without a reason.

Step 1: Send me your existing code

Please share these two files:

1. `main.py`
   - Open `main.py` in VS Code.
   - Copy the entire file and paste it here.
2. `game/game_engine.py`
   - Open `game_engine.py` .
   - Copy the entire file and paste it here.

Send both files in full, preferably in separate code blocks.

Step 2: What I'll do with your code

Once you've shared both files, we'll work through the following process:

Stage	What we'll do
1. Inspect	Understand the existing game loop, input handling, arm position, stamina, and AI logic.
2. Diagnose	Trace how alternating Left and Right key presses affect the arm position and identify why the push direction is inverted.
3. Explain	Show you the exact cause of the bug and explain the proposed fix before changing anything.
4. Fix	Implement the smallest necessary change and preserve existing functionality.

5. Test	Run the game and verify that player inputs move the arm toward the correct winning threshold.
6. Extend	Implement the AI surge cycle, visual warnings, exhaustion indicator, and counter-surge bonus.
7. Validate	Test stamina depletion and recovery, both winning conditions, and the R-key reset functionality.

Step 3: Our debugging rules

We'll follow three rules throughout the process:

- **Understand before changing:** No guessing about how your existing implementation works.
- **One logical change at a time:** Test each fix before introducing the next feature.
- **Preserve working code:** Keep your current project structure and avoid unnecessary rewrites.

We'll also make sure the counter-surge bonus interacts correctly with the AI's surge and cooldown states rather than triggering randomly.

Your turn, bro! Send me the complete contents of `main.py` and `game/game_engine.py`.

I'll inspect the actual implementation first, explain the inverted-push bug, and wait until we've established the cause before implementing the fix.

You said:

```
import sys
import pygame
from game.game_engine import GameEngine

SCREEN_WIDTH = 600
SCREEN_HEIGHT = 620
FPS = 60


def main():
    pygame.init()
    pygame.display.set_caption("Arm Wrestle Showdown")
    screen = pygame.display.set_mode((SCREEN_WIDTH,
SCREEN_HEIGHT))
```

```
    clock = pygame.time.Clock()

    engine = GameEngine(SCREEN_WIDTH, SCREEN_HEIGHT)

    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            else:
                engine.handle_event(event)

        engine.update()
        engine.render(screen)
        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()
    sys.exit()


if __name__ == "__main__":
    main()
```

game_engine.py is

```
import math
import random
import pygame


class GameEngine:

    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.arm_position = 0.0
        self.target_limit = 100.0
```

```
        self.last_key = None

        self.stamina = 100.0
        self.max_stamina = 100.0

        self.winner = None
        self.game_state = "PLAYING"
        self.ai_strength = 0.35

        self.font_big = pygame.font.SysFont(None, 44)
        self.font_med = pygame.font.SysFont(None, 26)

    def handle_event(self, event):
        if self.game_state != "PLAYING":
            if event.type == pygame.KEYDOWN and event.key ==
pygame.K_r:
                self.reset()
            return

        if event.type == pygame.KEYDOWN:
            if self.stamina <= 10:
                return

            if event.key == pygame.K_LEFT:
                if self.last_key != pygame.K_LEFT:
                    self.arm_position += 4.2
                    self.stamina = max(0.0, self.stamina - 2.0)
                    self.last_key = pygame.K_LEFT
            elif event.key == pygame.K_RIGHT:
                if self.last_key != pygame.K_RIGHT:
                    self.arm_position += 4.2
                    self.stamina = max(0.0, self.stamina - 2.0)
                    self.last_key = pygame.K_RIGHT

    def update(self):
        if self.game_state != "PLAYING":
            return

        ai_variance = random.uniform(0.3, 1.0)
        self.arm_position += self.ai_strength * ai_variance

        if self.stamina < self.max_stamina:
            self.stamina = min(self.max_stamina, self.stamina + 0.8)
```

```
            if self.arm_position <= -self.target_limit:
                self.winner = "PLAYER"
                self.game_state = "GAME_OVER"
            elif self.arm_position >= self.target_limit:
                self.winner = "COMPUTER"
                self.game_state = "GAME_OVER"

        def reset(self):
            self.arm_position = 0.0
            self.stamina = 100.0
            self.last_key = None
            self.winner = None
            self.game_state = "PLAYING"

        def render(self, screen):
            screen.fill((25, 28, 35))

            title_surf = self.font_big.render("ARM WRESTLE
SHOWDOWN", True, (240, 240, 240))
            screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2,
12))

            player_header = self.font_med.render("PLAYER", True, (80,
160, 255))
            computer_header = self.font_med.render("COMPUTER", True,
(255, 100, 80))
            screen.blit(player_header, (60, 55))
            screen.blit(computer_header, (self.width - 150, 55))

            table_rect = pygame.Rect(40, 100, self.width - 80, 310)
            pygame.draw.rect(screen, (110, 50, 15), table_rect,
border_radius=14)
            pygame.draw.rect(screen, (70, 30, 8), table_rect, width=5,
border_radius=14)

            pygame.draw.line(screen, (45, 18, 4), (self.width // 2, 100),
(self.width // 2, 410), 4)

            offset_x = (self.arm_position / self.target_limit) * 95
            hand_x = (self.width // 2) + int(offset_x)
            hand_y = 235
```

```
        p_shoulder = (70, 330)
        p_elbow = (140, 215)
        c_shoulder = (self.width - 70, 330)
        c_elbow = (self.width - 140, 215)

        pygame.draw.line(screen, (200, 145, 110), p_shoulder, p_elbow,
32)
        pygame.draw.line(screen, (215, 160, 125), p_elbow, (hand_x,
hand_y), 26)
        pygame.draw.circle(screen, (185, 130, 95), p_elbow, 18)

        pygame.draw.line(screen, (170, 110, 85), c_shoulder, c_elbow,
32)
        pygame.draw.line(screen, (185, 125, 95), c_elbow, (hand_x,
hand_y), 26)
        pygame.draw.circle(screen, (150, 95, 70), c_elbow, 18)

        pygame.draw.circle(screen, (225, 175, 140), (hand_x, hand_y),
24)
        pygame.draw.circle(screen, (160, 115, 85), (hand_x, hand_y),
24, width=3)

        stamina_label = self.font_med.render("STAMINA", True, (220,
220, 220))
        screen.blit(stamina_label, (40, 445))

        stamina_bg = pygame.Rect(140, 448, 240, 22)
        stamina_fill = pygame.Rect(140, 448, int(240 * (self.stamina /
self.max_stamina)), 22)
        pygame.draw.rect(screen, (45, 50, 60), stamina_bg,
border_radius=6)
        bar_color = (60, 210, 100) if self.stamina > 25 else (220, 60,
60)
        pygame.draw.rect(screen, bar_color, stamina_fill,
border_radius=6)

        if self.game_state == "GAME_OVER":
            overlay = pygame.Surface((self.width, self.height),
pygame.SRCALPHA)
            overlay.fill((0, 0, 0, 200))
            screen.blit(overlay, (0, 0))

            win_text = "PLAYER WINS THE MATCH!" if self.winner ==
```

```
"PLAYER" else "COMPUTER WINS!"
        color = (80, 240, 100) if self.winner == "PLAYER" else (240,
80, 80)
        text_surf = self.font_big.render(win_text, True, color)
        screen.blit(
            text_surf,
            (self.width // 2 - text_surf.get_width() // 2, self.height // 2 -
45)
        )

        restart_surf = self.font_med.render(
            "Press [R] to Rematch", True, (240, 240, 240)
        )
        screen.blit(
            restart_surf,
            (self.width // 2 - restart_surf.get_width() // 2, self.height //
2 + 10)
        )
```

ChatGPT said:

BRO 😭 okay, I've gone through both files. We have found the main bug, and there's a second issue in the input logic that we should address at the same time.

I'm not going to rewrite anything yet. First, let's understand what's happening and agree on the fix.

Step 1: Diagnose the inverted push bug

1. The main problem is in `handle_event()`

Your current code has this:

```
if event.key == pygame.K_LEFT:
    if self.last_key != pygame.K_LEFT:
        self.arm_position += 4.2
        self.stamina = max(0.0, self.stamina - 2.0)
        self.last_key = pygame.K_LEFT

elif event.key == pygame.K_RIGHT:
    if self.last_key != pygame.K_RIGHT:
        self.arm_position += 4.2
        self.stamina = max(0.0, self.stamina - 2.0)
        self.last_key = pygame.K_RIGHT
```

Can you spot the problem? Both Left and Right increase `arm_position` by `4.2`.

Your game uses these winning conditions:

```
if self.arm_position <= -self.target_limit:
    self.winner = "PLAYER"
    self.game_state = "GAME_OVER"

elif self.arm_position >= self.target_limit:
    self.winner = "COMPUTER"
    self.game_state = "GAME_OVER"
```

This means:

- `arm_position = -100` : Player wins.
- `arm_position = +100` : Computer wins.
- `arm_position = 0` : Starting position.

But every successful player input moves the arm in the positive direction, toward the computer's winning threshold.

That's why your player accidentally helps the computer win!

2. There's another problem in `update()`

Your AI currently applies force like this:

```
ai_variance = random.uniform(0.3, 1.0)
self.arm_position += self.ai_strength * ai_variance
```

This also increases `arm_position`, meaning the AI pushes toward its own winning threshold.

That part is consistent with the computer winning at `+100`. However, the player needs to apply force in the opposite direction.

One additional observation: your current AI applies force on every update, regardless of whether the player is exhausted. That's fine for the basic game, but we'll need to consider the interaction when implementing stamina recovery and AI cooldown.

Step 2: The fix I propose

We should make the smallest change necessary to correct the direction.

Fix A: Correct the player push direction

The intended behavior is:

Player input	Arm position change	Effect
Left arrow	`−4.2`	Pushes toward the player's winning threshold
Right arrow	`−4.2`	Pushes toward the player's winning threshold
Computer force	Positive	Pushes toward the computer's winning threshold

The relevant change would be:

```
self.arm_position −= 4.2
```

We would apply that change to both arrow-key branches.

Why not change the winning thresholds instead?

Because your existing thresholds are internally consistent: negative values mean a player victory, and positive values mean a computer victory. Reversing them would introduce unnecessary changes to the game-over logic.

Fix B: Preserve alternating-key behavior

You currently have:

```
if self.last_key != pygame.K_LEFT:
```

and:

```
if self.last_key != pygame.K_RIGHT:
```

These conditions mean a player receives a push only when pressing a different key from the previous accepted key.

For example:

1. Press Left → push.
2. Press Left again → no push.
3. Press Right → push.
4. Press Left → push.

This is consistent with your alternating-input requirement, so **we should preserve this behavior**.

Fix C: Address the stamina threshold

Your current condition is:

```
if self.stamina <= 10:
    return
```

This blocks player input when stamina reaches 10 or below.

Your `update()` method also regenerates stamina:

```
if self.stamina < self.max_stamina:
    self.stamina = min(self.max_stamina, self.stamina + 0.8)
```

That means stamina recovery is already implemented.

We should preserve this mechanism initially and later verify that input resumes correctly after stamina recovers above the threshold.

Step 3: What about the other three tasks?

I've also checked your current implementation against the README requirements you provided.

Requirement	Current status	Proposed approach
Correct player push direction	Bug present	Reverse the direction of player input
Dynamic AI surge cycle	Not implemented	Introduce timed surge and cooldown states
Visual surge warning	Not implemented	Display a warning during the AI surge
Player exhaustion indicator	Not implemented	Display a warning when stamina is low
Counter-surge bonus	Not implemented	Detect the transition from surge to cooldown and temporarily boost the player's push and recovery
Stamina depletion and recovery	Partially implemented	Preserve the current mechanism and test its behavior
Player and computer win conditions	Implemented	Keep the existing thresholds
Restart using R	Implemented	Verify that all newly introduced states reset correctly

One important design decision for the counter-surge bonus: we should trigger it when the AI transitions from its surge into cooldown, rather than whenever the player presses a key during an arbitrary period.

That makes the bonus predictable and testable.

Also, your current `reset()` method doesn't explicitly restore `ai_strength`, although it doesn't change anywhere in the current implementation. Once we introduce surge states, we'll need to reset those states and their timers as well.

Step 4: Your first change

Before we introduce any new features, I recommend fixing only the inverted-push bug.

The proposed change is just two lines: change the player input's `arm_position += 4.2` to `arm_position -= 4.2` in both arrow-key branches.

I haven't changed your files or implemented the fix yet.

Are you okay with this fix? If you confirm, we'll implement it, run the game, and verify that alternating the arrow keys moves the arm toward `-100` before moving on to the AI surge cycle.

You said:

give me the fixed file content fully so i can replace it

ChatGPT said:

Bro, replace your entire `game/game_engine.py` file with the code below. I've fixed the inverted push bug while preserving your existing game logic, stamina system, rendering, win conditions, and restart functionality.

No changes are needed to `main.py` for this fix.

`game/game_engine.py` — Complete fixed code



Step 2: Save and run the game

1. Open `game/game_engine.py` in VS Code.
2. Select all the existing code.
3. Replace it with the complete code above.
4. Save the file using **Cmd + S**.
5. Open your terminal in the `arm_wrestle` project folder.
6. Run:

```
python3 main.py
```

Step 3: Test the fix

Try alternating the Left and Right arrow keys.

You should observe the following:

- **Left → Right → Left → Right:** The player pushes the arm toward the player's winning threshold.
- **No input:** The computer gradually pushes the arm toward its own winning threshold.
- **Stamina at or below 10:** Player input is blocked until stamina recovers.
- **Arm reaches -100:** The player wins.
- **Arm reaches +100:** The computer wins.
- **Press R after Game Over:** The game resets.

One important note: This fixes the inverted push direction only. The dynamic AI surge, visual warnings, exhaustion indicator, and counter-surge bonus are still pending.

Run the game and tell me whether the arm now moves in the correct direction. We'll test this fix before implementing the next feature.

You said:

Hello from the pygame community.
https://www.pygame.org/contribute.html

```
Traceback (most recent call last):
  File "/Users/nishitha/Documents/SE-LAB_1/LAB4_VibeCoding/53_arm_wrestle/main.py", line 3, in <module>
    from game.game_engine import GameEngine
  File "/Users/nishitha/Documents/SE-LAB_1/LAB4_VibeCoding/53_arm_wrestle/game/game_engine.py", line 7
    def __init__(self, width, height):
    ^
IndentationError: expected an indented block after class definition on line 5
```

ChatGPT said:

Bro 😭 this error means Python sees `class GameEngine:` but the code inside the class isn't indented correctly. This can happen when copying and pasting the file.

Let's fix it properly.

Step 1: Replace the entire `game/game_engine.py` file

Open `game/game_engine.py` in VS Code, press **Cmd + A**, delete everything, and paste this complete code.



Step 2: Save the file

Press **Cmd + S**.

Make sure you replace the entire file, including the `class GameEngine:` line. Don't paste the new code inside the old code.

Step 3: Run the game again

In your terminal, run:

```
python3 main.py
```

Run this command from your project directory:

```
53_arm_wrestle
```

The Pygame community greeting is normal; it isn't the error.

If the same indentation error appears again, send me a screenshot of the first 20 lines of `game_engine.py` in VS Code. We'll identify exactly what's wrong.

You said:

```
pygame 2.6.1 (SDL 2.28.4, Python 3.11.9)
Hello from the pygame community.
https://www.pygame.org/contribute.html
Traceback (most recent call last):
  File "/Users/nishitha/Documents/SE-LAB_1/LAB4_VibeCoding/53_arm_wrestle/main.py", line 3, in <module>
    from game.game_engine import GameEngine
  File "/Users/nishitha/Documents/SE-LAB_1/LAB4_VibeCoding/53_arm_wrestle/game/game_engine.py", line 5
```

```
    def init(self, width, height):
    ^
IndentationError: expected an indented block after class definition on line 4
```

ChatGPT said:

BRO 😭 I found the issue in your error message.

Your file currently has:

```
def init(self, width, height):
```

But it must be:

```
def __init__(self, width, height):
```

There are **two underscores before `init` and two after it**. However, the error also indicates an indentation problem, so let's fix both by replacing the entire file.

Step 1: Replace `game/game_engine.py` completely

Open the file in VS Code, press **Cmd + A**, and replace everything with the code below.



Step 2: Check the first four lines carefully

After pasting, the beginning of the file must look **exactly** like this:

```
import random
import pygame


class GameEngine:
    def __init__(self, width, height):
```

Notice:

- `class GameEngine:` starts at the far left.
- `def __init__` has **four spaces** before it.
- The constructor has two underscores before and after `init`.

Step 3: Save and run

Press **Cmd + S**, then run:

```
python3 main.py
```

Make sure your terminal is in the `53_arm_wrestle` directory.

If it still fails, don't replace the code again. Send me the first 10 lines of your actual `game_engine.py` file, copied directly from VS Code. That will help us determine why the code is changing when you paste it.

You said:

same error

ChatGPT said:

Bro, don't paste the code again just yet. If you're getting the **same error**, we need to check what Python is actually reading. The traceback says `def init(self, width, height):` , which is different from the code I gave you.

Step 1: Run these commands in your terminal

Copy and run this entire block:

```
cd "/Users/nishitha/Documents/SE-LAB_1/LAB4_VibeCoding/53_arm_wrestle"

pwd

head -n 15 game/game_engine.py
```

Step 2: Send me the output

This will tell us:

- Whether you're running the correct project folder.
- What the first 15 lines of your actual `game_engine.py` file contain.
- Whether the constructor is incorrectly named or indented.

Don't modify anything else yet. Send me the terminal output, and I'll identify the exact problem so we can fix it without repeating the same steps.

You said:

```
import random
import pygame

class GameEngine:
def init(self, width, height):
self.width = width
self.height = height

    self.arm_position = 0.0
    self.target_limit = 100.0
    self.last_key = None
```

self.stamina = 100.0
self.max_stamina = 100.0
self.winner = None
self.game_state = "PLAYING"

ChatGPT said:

BRO 😭 FOUND IT. Your file has **two problems**:

1. `def init(self, width, height):` should be `def __init__(self, width, height):`.
2. The code inside `class GameEngine` and the constructor isn't indented correctly. Python requires indentation to define which code belongs to the class and its methods.

Let's fix this properly.

Step 1: Replace the entire `game/game_engine.py` file

Open `game/game_engine.py` in VS Code, press **Cmd + A**, and replace everything with this complete code.

Important: Copy from the first line to the last line. Don't leave any of the old code in the file.

Step 2: Save and verify the file

1. Press **Cmd + S**.
2. In the terminal, run:

```
head -n 10 game/game_engine.py
```

The output must begin exactly like this:

```
import random
import pygame


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
```

Step 3: Run the game

Once those lines are correct, execute:

```
python3 -m py_compile game/game_engine.py
```

If the command returns no output, Python has successfully compiled the file.

Then run:

```
python3 main.py
```

Do the verification step first, bro. We've identified the exact issue in your actual file; there's no need to keep guessing or repeatedly running broken code.

You said:

Sorry: IndentationError: expected an indented block after class definition on line 4 (game_engine.py, line 5)%

ChatGPT said:

Bro 😭 your file still isn't being indented correctly. Let's fix it directly through the terminal instead of copying code into VS Code again.

Step 1: Run this command

Copy and paste this entire command into your terminal:

```
cd "/Users/nishitha/Documents/SE-LAB_1/LAB4_VibeCoding/53_arm_wrestle"

python3 -c "from pathlib import Path; p=Path('game/game_engine.py'); s=p.read_text(); print(repr(s[:250]))"
```

This displays the actual characters at the beginning of your file, including indentation.

Step 2: Send me the output

I want to check whether:

- Your editor is removing indentation when pasting.
- The file contains unexpected characters.
- You're editing a different file from the one Python is executing.

Don't paste the full code again yet. Send me the output of that command, and we'll fix the underlying problem rather than repeating the same steps.

You said:

python3 -c "from pathlib import Path;

p=Path('game/game_engine.py'); s=p.read_text();
print(repr(s[:250]))"
'import random\nimport pygame\n\nclass GameEngine:\ndef
init(self, width, height):\nself.width = width\nself.height = height\n\n
self.arm_position = 0.0\n self.target_limit = 100.0\n self.last_key
= None\n self.stamina = 100.0\n self.max_stamina = '

ChatGPT said:

BRO 😭 FOUND THE EXACT PROBLEM. Your file still contains `def init` without underscores, and the lines inside the class have no indentation.

Let's stop fighting with copy-paste. **We'll fix the file directly from your terminal.** This will replace the entire file with correctly formatted code.

Step 1: Copy and run this entire command

Make sure your terminal is in the `53_arm_wrestle` folder.

Paste this complete block into your terminal:

```
cat > game/game_engine.py <<'PYTHON'
import random
import pygame


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.arm_position = 0.0
        self.target_limit = 100.0
        self.last_key = None

        self.stamina = 100.0
        self.max_stamina = 100.0

        self.winner = None
        self.game_state = "PLAYING"
        self.ai_strength = 0.35

        self.font_big = pygame.font.SysFont(None, 44)
        self.font_med = pygame.font.SysFont(None, 26)
```

```
    def handle_event(self, event):
        if self.game_state != "PLAYING":
            if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                self.reset()
            return

        if event.type == pygame.KEYDOWN:
            if self.stamina <= 10:
                return

            if event.key == pygame.K_LEFT:
                if self.last_key != pygame.K_LEFT:
                    self.arm_position -= 4.2
                    self.stamina = max(0.0, self.stamina - 2.0)
                    self.last_key = pygame.K_LEFT

            elif event.key == pygame.K_RIGHT:
                if self.last_key != pygame.K_RIGHT:
                    self.arm_position -= 4.2
                    self.stamina = max(0.0, self.stamina - 2.0)
                    self.last_key = pygame.K_RIGHT

    def update(self):
        if self.game_state != "PLAYING":
            return

        ai_variance = random.uniform(0.3, 1.0)
        self.arm_position += self.ai_strength * ai_variance

        if self.stamina < self.max_stamina:
            self.stamina = min(
                self.max_stamina,
                self.stamina + 0.8
            )

        if self.arm_position <= -self.target_limit:
            self.winner = "PLAYER"
            self.game_state = "GAME_OVER"

        elif self.arm_position >= self.target_limit:
```

```
            self.winner = "COMPUTER"
            self.game_state = "GAME_OVER"

    def reset(self):
        self.arm_position = 0.0
        self.stamina = 100.0
        self.last_key = None
        self.winner = None
        self.game_state = "PLAYING"

    def render(self, screen):
        screen.fill((25, 28, 35))

        title_surf = self.font_big.render(
            "ARM WRESTLE SHOWDOWN", True, (240, 240, 240)
        )
        screen.blit(
            title_surf,
            (self.width // 2 - title_surf.get_width() // 2, 12)
        )

        player_header = self.font_med.render(
            "PLAYER", True, (80, 160, 255)
        )
        computer_header = self.font_med.render(
            "COMPUTER", True, (255, 100, 80)
        )
        screen.blit(player_header, (60, 55))
        screen.blit(computer_header, (self.width - 150, 55))

        table_rect = pygame.Rect(40, 100, self.width - 80, 310)
        pygame.draw.rect(
            screen, (110, 50, 15), table_rect, border_radius=14
        )
        pygame.draw.rect(
            screen, (70, 30, 8), table_rect, width=5, border_radius=14
        )

        pygame.draw.line(
            screen,
            (45, 18, 4),
```

```
            (self.width // 2, 100),
            (self.width // 2, 410),
            4
        )

        offset_x = (self.arm_position / self.target_limit) * 95
        hand_x = (self.width // 2) + int(offset_x)
        hand_y = 235

        p_shoulder = (70, 330)
        p_elbow = (140, 215)
        c_shoulder = (self.width - 70, 330)
        c_elbow = (self.width - 140, 215)

        pygame.draw.line(
            screen, (200, 145, 110), p_shoulder, p_elbow, 32
        )
        pygame.draw.line(
            screen, (215, 160, 125), p_elbow, (hand_x, hand_y), 26
        )
        pygame.draw.circle(screen, (185, 130, 95), p_elbow, 18)

        pygame.draw.line(
            screen, (170, 110, 85), c_shoulder, c_elbow, 32
        )
        pygame.draw.line(
            screen, (185, 125, 95), c_elbow, (hand_x, hand_y), 26
        )
        pygame.draw.circle(screen, (150, 95, 70), c_elbow, 18)

        pygame.draw.circle(screen, (225, 175, 140), (hand_x, hand_y), 24)
        pygame.draw.circle(
            screen, (160, 115, 85), (hand_x, hand_y), 24, width=3
        )

        stamina_label = self.font_med.render(
            "STAMINA", True, (220, 220, 220)
        )
        screen.blit(stamina_label, (40, 445))

        stamina_bg = pygame.Rect(140, 448, 240, 22)
```

```
        stamina_fill = pygame.Rect(
            140,
            448,
            int(240 * (self.stamina / self.max_stamina)),
            22
        )

        pygame.draw.rect(
            screen, (45, 50, 60), stamina_bg, border_radius=6
        )

        bar_color = (
            (60, 210, 100)
            if self.stamina > 25
            else (220, 60, 60)
        )
        pygame.draw.rect(
            screen, bar_color, stamina_fill, border_radius=6
        )

        if self.game_state == "GAME_OVER":
            overlay = pygame.Surface(
                (self.width, self.height), pygame.SRCALPHA
            )
            overlay.fill((0, 0, 0, 200))
            screen.blit(overlay, (0, 0))

            win_text = (
                "PLAYER WINS THE MATCH!"
                if self.winner == "PLAYER"
                else "COMPUTER WINS!"
            )
            color = (
                (80, 240, 100)
                if self.winner == "PLAYER"
                else (240, 80, 80)
            )

            text_surf = self.font_big.render(win_text, True, color)
            screen.blit(
                text_surf,
```

```
                (
                    self.width // 2 - text_surf.get_width() // 2,
                    self.height // 2 - 45
                )
            )

            restart_surf = self.font_med.render(
                "Press [R] to Rematch", True, (240, 240, 240)
            )
            screen.blit(
                restart_surf,
                (
                    self.width // 2 - restart_surf.get_width() // 2,
                    self.height // 2 + 10
                )
            )
```

PYTHON

Important: The final `PYTHON` must be on its own line, with no spaces before or after it. That tells your terminal when to finish writing the file.

Step 2: Verify the fix

Now run:

```
python3 -m py_compile game/game_engine.py
```

If it returns to the terminal without an error, the indentation issue is fixed.

Step 3: Run the game

```
python3 main.py
```

Your player should now push the arm toward `-100`, while the computer pushes it toward `+100`.

Tell me what happens after Step 2. We won't move on to the AI surge feature until the game runs correctly.

You said:

the game opened

ChatGPT said:

BROOO 😭🔥 FINALLY! The game is running!

We've fixed the indentation issue and the inverted push direction. Now let's verify the gameplay before adding new features.

Step 1: Test these 4 things

Test 1 — Player movement

- Alternate between the Left and Right arrow keys rapidly.
- The arm should move toward the **player's winning threshold (-100)**.
- Stop pressing keys and observe whether the computer pushes back.

Test 2 — Stamina

- Keep alternating the arrow keys.
- Watch the stamina bar.
- Check whether stamina decreases and recovers when you stop pressing keys.

Test 3 — Computer wins

- Let the computer push without interfering.
- Check whether the game ends when the arm reaches the computer's winning threshold (+100).

Test 4 — Restart

- After Game Over, press `R`.
- The arm should return to the centre, and stamina should reset to 100.

Step 2: Tell me the results

Just reply in this format:

- Player movement: Working / Not working
- Stamina: Working / Not working
- Computer wins: Working / Not working
- Restart: Working / Not working

Once we've verified these, we'll implement the **dynamic AI surge cycle** as our next feature, test it, and then move on to the visual warnings and counter-surge bonus.

One step at a time, bro. 💪

You said:

ChatGPT said:

BROOO 😭🔥 **THE GAME IS RUNNING, AND THE PLAYER HAS WON!**

From your screenshot, I can confirm:

- ✅ The game launches successfully.
- ✅ The Game Over screen renders correctly.
- ✅ The player victory message appears.
- ✅ The restart prompt is displayed.
- ✅ The stamina bar renders.

However, the screenshot alone doesn't tell us whether the player won through normal gameplay or whether the arm reached the threshold correctly.

Step 1: Test the restart

1. Press **R** on your keyboard.
2. Check whether the game returns to the starting screen.
3. Confirm that the arm returns to the centre and stamina resets to 100.

Step 2: Test the computer's behaviour

After restarting, don't press any keys. Let the computer push the arm.

Check whether the computer eventually wins and displays **COMPUTER WINS!**

Step 3: Tell me what happens

Once we confirm these two behaviours, we'll move to the next requirement from your lab:

Dynamic AI Surge Cycle 💪

We'll implement periodic high-power AI surges followed by reduced resistance during cooldown, while preserving the working game you have now.

One feature at a time, bro!

You said:

yes the computer won and stamina went to 100

yes the computer won and stamina went to 100

ChatGPT said:

BROOO 🔥💪 We've got the basic game working!

✅ Current progress

Requirement	Status
Game launches without errors	✅
Player pushes toward their winning threshold	✅
Computer pushes toward its winning threshold	✅
Computer victory detection	✅
Stamina bar and recovery	✅
Restart using R	✅

Now let's implement the next feature **without breaking what we've already fixed**.

Step 2: Implement the Dynamic AI Surge Cycle

Your lab requires the computer to alternate between two states:

1. 🔴 POWER SURGE

- The computer applies significantly stronger force.
- The surge lasts for a limited duration.
- We'll add a visual warning in a later step.

2. 🟢 COOLDOWN

- The computer's force drops below its normal strength.
- The player gets an opportunity to push back.
- We'll use the transition into cooldown for the counter-surge bonus later.

How we'll implement it

We'll introduce a few new variables in `GameEngine`:

- `ai_state`: Tracks whether the AI is surging or cooling down.

- `ai_state_timer` : Tracks how long the current state has lasted.
- `ai_surge_duration` : Controls the length of a surge.
- `ai_cooldown_duration` : Controls the length of the cooldown.
- `ai_strength` : Determines how much force the computer applies.

We'll use Pygame's frame timing to manage the cycle, rather than relying on arbitrary delays that freeze the game.

Before we change the code

One quick question, bro:

How difficult should the game feel?

A. Easy: Short surges, long cooldowns, weaker AI.

B. Balanced: Moderate surges and cooldowns, with a fair chance for the player to win.

C. Hard: Strong surges, short cooldowns, and an aggressive computer.

I'd recommend **B — Balanced** for your lab demonstration.

You said:

okay

ChatGPT said:

Okay bro 😭💪 Let's implement the **Balanced Dynamic AI Surge Cycle** now.

We'll replace the entire `game/game_engine.py` file. This version preserves the working game and adds the AI's three states: normal, power surge, and cooldown.

Step 1: Replace `game/game_engine.py`

Open the file in VS Code, press **Cmd + A**, and replace everything with this complete code.

```
import random
import pygame


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
```

```
        # Arm position and winning threshold
        self.arm_position = 0.0
        self.target_limit = 100.0
        self.last_key = None

        # Player stamina
        self.stamina = 100.0
        self.max_stamina = 100.0

        # Game state
        self.winner = None
        self.game_state = "PLAYING"

        # AI configuration
        self.ai_strength = 0.35
        self.ai_state = "NORMAL"
        self.ai_state_timer = 0.0

        # AI cycle durations in seconds
        self.ai_normal_duration = 5.0
        self.ai_surge_duration = 2.5
        self.ai_cooldown_duration = 4.0

        # AI force multipliers
        self.ai_surge_multiplier = 1.8
        self.ai_cooldown_multiplier = 0.4

        # Track elapsed time between updates
        self.last_update_time = pygame.time.get_ticks()

        # Fonts
        self.font_big = pygame.font.SysFont(None, 44)
        self.font_med = pygame.font.SysFont(None, 26)

    def handle_event(self, event):
        # Restart after Game Over
        if self.game_state != "PLAYING":
            if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                self.reset()
            return
```

```
        if event.type == pygame.KEYDOWN:
            # Block player input when stamina is too low
            if self.stamina <= 10:
                return

            # Alternate between Left and Right
            if event.key == pygame.K_LEFT:
                if self.last_key != pygame.K_LEFT:
                    # Player pushes toward -100
                    self.arm_position -= 4.2

                    self.stamina = max(
                        0.0,
                        self.stamina - 2.0
                    )

                    self.last_key = pygame.K_LEFT

            elif event.key == pygame.K_RIGHT:
                if self.last_key != pygame.K_RIGHT:
                    # Player pushes toward -100
                    self.arm_position -= 4.2

                    self.stamina = max(
                        0.0,
                        self.stamina - 2.0
                    )

                    self.last_key = pygame.K_RIGHT

    def update_ai_state(self, dt):
        """
        Manage the AI's normal, surge, and cooldown states.
        """

        self.ai_state_timer += dt

        if self.ai_state == "NORMAL":
            if self.ai_state_timer >= self.ai_normal_duration:
                self.ai_state = "SURGE"
                self.ai_state_timer = 0.0
```

```
        elif self.ai_state == "SURGE":
            if self.ai_state_timer >= self.ai_surge_duration:
                self.ai_state = "COOLDOWN"
                self.ai_state_timer = 0.0

        elif self.ai_state == "COOLDOWN":
            if self.ai_state_timer >= self.ai_cooldown_duration:
                self.ai_state = "NORMAL"
                self.ai_state_timer = 0.0

    def update(self):
        if self.game_state != "PLAYING":
            return

        # Calculate elapsed time for the AI state timer
        current_time = pygame.time.get_ticks()

        dt = (current_time - self.last_update_time) / 1000.0
        self.last_update_time = current_time

        # Avoid unusually large timer jumps after a pause
        dt = min(dt, 0.1)

        # Update the AI's current state
        self.update_ai_state(dt)

        # Determine AI force for the current state
        if self.ai_state == "SURGE":
            current_ai_strength = (
                self.ai_strength * self.ai_surge_multiplier
            )

        elif self.ai_state == "COOLDOWN":
            current_ai_strength = (
                self.ai_strength * self.ai_cooldown_multiplier
            )

        else:
            current_ai_strength = self.ai_strength
```

```
        # Computer applies opposing force
        ai_variance = random.uniform(0.3, 1.0)

        self.arm_position += (
            current_ai_strength * ai_variance
        )

        # Recover player stamina gradually
        if self.stamina < self.max_stamina:
            self.stamina = min(
                self.max_stamina,
                self.stamina + 0.8
            )

        # Check winning conditions
        if self.arm_position <= -self.target_limit:
            self.winner = "PLAYER"
            self.game_state = "GAME_OVER"

        elif self.arm_position >= self.target_limit:
            self.winner = "COMPUTER"
            self.game_state = "GAME_OVER"

    def reset(self):
        # Reset arm and stamina
        self.arm_position = 0.0
        self.stamina = 100.0
        self.last_key = None

        # Reset game state
        self.winner = None
        self.game_state = "PLAYING"

        # Reset AI cycle
        self.ai_state = "NORMAL"
        self.ai_state_timer = 0.0

        # Reset timing
        self.last_update_time = pygame.time.get_ticks()

    def render(self, screen):
```

```
        screen.fill((25, 28, 35))

        # Title
        title_surf = self.font_big.render(
            "ARM WRESTLE SHOWDOWN",
            True,
            (240, 240, 240)
        )

        screen.blit(
            title_surf,
            (
                self.width // 2 - title_surf.get_width() // 2,
                12
            )
        )

        # Player and computer labels
        player_header = self.font_med.render(
            "PLAYER",
            True,
            (80, 160, 255)
        )

        computer_header = self.font_med.render(
            "COMPUTER",
            True,
            (255, 100, 80)
        )

        screen.blit(player_header, (60, 55))
        screen.blit(
            computer_header,
            (self.width - 150, 55)
        )

        # Table
        table_rect = pygame.Rect(
            40,
            100,
            self.width - 80,
```

```
        310
    )

    pygame.draw.rect(
        screen,
        (110, 50, 15),
        table_rect,
        border_radius=14
    )

    pygame.draw.rect(
        screen,
        (70, 30, 8),
        table_rect,
        width=5,
        border_radius=14
    )

    pygame.draw.line(
        screen,
        (45, 18, 4),
        (self.width // 2, 100),
        (self.width // 2, 410),
        4
    )

    # Arm position
    offset_x = (
        self.arm_position / self.target_limit
    ) * 95

    hand_x = (
        self.width // 2
    ) + int(offset_x)

    hand_y = 235

    p_shoulder = (70, 330)
    p_elbow = (140, 215)

    c_shoulder = (self.width - 70, 330)
```

```
        c_elbow = (self.width - 140, 215)

        # Player arm
        pygame.draw.line(
            screen,
            (200, 145, 110),
            p_shoulder,
            p_elbow,
            32
        )

        pygame.draw.line(
            screen,
            (215, 160, 125),
            p_elbow,
            (hand_x, hand_y),
            26
        )

        pygame.draw.circle(
            screen,
            (185, 130, 95),
            p_elbow,
            18
        )

        # Computer arm
        pygame.draw.line(
            screen,
            (170, 110, 85),
            c_shoulder,
            c_elbow,
            32
        )

        pygame.draw.line(
            screen,
            (185, 125, 95),
            c_elbow,
            (hand_x, hand_y),
            26
```

```
        )

        pygame.draw.circle(
            screen,
            (150, 95, 70),
            c_elbow,
            18
        )

        # Clasped hands
        pygame.draw.circle(
            screen,
            (225, 175, 140),
            (hand_x, hand_y),
            24
        )

        pygame.draw.circle(
            screen,
            (160, 115, 85),
            (hand_x, hand_y),
            24,
            width=3
        )

        # Stamina label
        stamina_label = self.font_med.render(
            "STAMINA",
            True,
            (220, 220, 220)
        )

        screen.blit(stamina_label, (40, 445))

        # Stamina bar
        stamina_bg = pygame.Rect(
            140,
            448,
            240,
            22
        )
```

```
        stamina_fill = pygame.Rect(
            140,
            448,
            int(
                240 * (
                    self.stamina / self.max_stamina
                )
            ),
            22
        )

        pygame.draw.rect(
            screen,
            (45, 50, 60),
            stamina_bg,
            border_radius=6
        )

        bar_color = (
            (60, 210, 100)
            if self.stamina > 25
            else (220, 60, 60)
        )

        pygame.draw.rect(
            screen,
            bar_color,
            stamina_fill,
            border_radius=6
        )

        # Game Over screen
        if self.game_state == "GAME_OVER":
            overlay = pygame.Surface(
                (self.width, self.height),
                pygame.SRCALPHA
            )

            overlay.fill((0, 0, 0, 200))
            screen.blit(overlay, (0, 0))
```

```
win_text = (
    "PLAYER WINS THE MATCH!"
    if self.winner == "PLAYER"
    else "COMPUTER WINS!"
)

color = (
    (80, 240, 100)
    if self.winner == "PLAYER"
    else (240, 80, 80)
)

text_surf = self.font_big.render(
    win_text,
    True,
    color
)

screen.blit(
    text_surf,
    (
        self.width // 2 - text_surf.get_width() // 2,
        self.height // 2 - 45
    )
)

restart_surf = self.font_med.render(
    "Press [R] to Rematch",
    True,
    (240, 240, 240)
)

screen.blit(
    restart_surf,
    (
        self.width // 2 - restart_surf.get_width() // 2,
        self.height // 2 + 10
    )
)
```

Step 2: Save and test

Save the file using **Cmd + S**.

First, check for syntax errors:

```
python3 -m py_compile game/game_engine.py
```

If no error appears, run:

```
python3 main.py
```

You don't need to change `main.py`.

Step 3: Verify the AI cycle

The AI now follows this repeating cycle:

State	Duration	AI behaviour
Normal	5 seconds	Applies normal force
Power surge	2.5 seconds	Applies 1.8× normal force
Cooldown	4 seconds	Applies 0.4× normal force
Repeat	—	Returns to normal

The cycle repeats automatically while the game is running.

What to test

1. Start the game and alternate the arrow keys.
2. Keep playing for at least 12 seconds.
3. Observe whether the computer pushes harder during the surge and becomes weaker during cooldown.
4. Allow the computer to win, then press `R`.
5. Verify that the arm and stamina reset and the AI returns to its normal state.

Important: This step implements the AI's behaviour, but the visual surge warning, exhaustion indicator, and counter-surge bonus are not implemented yet. We'll add those after verifying this cycle works.

Tell me whether the game runs successfully and whether you can notice the difference in the computer's strength.

You said:

yes i cant notice it

ChatGPT said:

Bro, that's because we have two problems with how the surge is presented. 😭

1. **There's no visual indicator yet.** The AI changes strength internally, but the screen doesn't tell you when a surge begins.
2. **The strength difference is difficult to judge.** The current multipliers are 1.8× during a surge and 0.4× during cooldown. Since the arm moves gradually, the difference isn't immediately obvious.

Let's fix this so you can clearly demonstrate the AI cycle during your lab evaluation.

Step 1: Make the AI cycle noticeable

We'll make three improvements:

- 🔴 **POWER SURGE:** The AI applies 2.5× its normal force.
- 🟢 **COOLDOWN:** The AI applies only 0.25× its normal force.
- ⚪ **NORMAL:** The AI applies its usual force.

We'll also display the current AI state on the screen, along with a countdown timer.

This adds a visible indicator now, even though we originally planned to implement the visual warnings in the next stage.

Step 2: First, let's verify your current implementation

Before replacing the file again, run this command from your project directory:

```
grep -n -E "ai_surge_multiplier|ai_cooldown_multiplier|ai_state|def render" game/game_engine.py
```

You should see the AI state variables, multipliers, and rendering method.

Send me the output. I'll use your actual implementation to give you the complete corrected `game_engine.py` file, with the visible AI state indicator and more noticeable force differences, without disturbing the working player movement, stamina, or restart functionality.

You said:

```
25:         self.ai_state = "NORMAL"
26:         self.ai_state_timer = 0.0
```

```
34:       self.ai_surge_multiplier = 1.8
35:       self.ai_cooldown_multiplier = 0.4
81:   def update_ai_state(self, dt):
86:       self.ai_state_timer += dt
88:       if self.ai_state == "NORMAL":
89:           if self.ai_state_timer >= self.ai_normal_duration:
90:               self.ai_state = "SURGE"
91:               self.ai_state_timer = 0.0
93:       elif self.ai_state == "SURGE":
94:           if self.ai_state_timer >= self.ai_surge_duration:
95:               self.ai_state = "COOLDOWN"
96:               self.ai_state_timer = 0.0
98:       elif self.ai_state == "COOLDOWN":
99:           if self.ai_state_timer >= self.ai_cooldown_duration:
100:              self.ai_state = "NORMAL"
101:              self.ai_state_timer = 0.0
117:      self.update_ai_state(dt)
120:      if self.ai_state == "SURGE":
122:              self.ai_strength * self.ai_surge_multiplier
125:      elif self.ai_state == "COOLDOWN":
127:              self.ai_strength * self.ai_cooldown_multiplier
167:      self.ai_state = "NORMAL"
168:      self.ai_state_timer = 0.0
173:  def render(self, screen):
```

ChatGPT said:

BRO perfect 😭🔥 Your AI cycle is already implemented correctly. The issue is that **the game doesn't visually communicate the AI's current state**, and the force differences aren't dramatic enough.

Let's fix both. We'll add a clear state indicator, a countdown timer, and stronger differences between the AI's normal, surge, and cooldown phases.

Step 1: Replace `game/game_engine.py` completely

Open the file in VS Code, press **Cmd + A**, and replace everything with this code.

I've preserved the player movement, stamina system, winning conditions, and restart functionality.

```
import random
import pygame


class GameEngine:
```

```
    def __init__(self, width, height):
        self.width = width
        self.height = height

        # Arm position
        self.arm_position = 0.0
        self.target_limit = 100.0
        self.last_key = None

        # Player stamina
        self.stamina = 100.0
        self.max_stamina = 100.0

        # Game state
        self.winner = None
        self.game_state = "PLAYING"

        # AI configuration
        self.ai_strength = 0.35
        self.ai_state = "NORMAL"
        self.ai_state_timer = 0.0

        # AI cycle durations
        self.ai_normal_duration = 5.0
        self.ai_surge_duration = 2.5
        self.ai_cooldown_duration = 4.0

        # More noticeable AI strength differences
        self.ai_surge_multiplier = 2.5
        self.ai_cooldown_multiplier = 0.25

        # Timing
        self.last_update_time = pygame.time.get_ticks()

        # Fonts
        self.font_big = pygame.font.SysFont(None, 44)
        self.font_med = pygame.font.SysFont(None, 26)
        self.font_small = pygame.font.SysFont(None, 22)

    def handle_event(self, event):
        if self.game_state != "PLAYING":
```

```
            if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                self.reset()
            return

        if event.type == pygame.KEYDOWN:
            if self.stamina <= 10:
                return

            if event.key == pygame.K_LEFT:
                if self.last_key != pygame.K_LEFT:
                    self.arm_position -= 4.2
                    self.stamina = max(
                        0.0,
                        self.stamina - 2.0
                    )
                    self.last_key = pygame.K_LEFT

            elif event.key == pygame.K_RIGHT:
                if self.last_key != pygame.K_RIGHT:
                    self.arm_position -= 4.2
                    self.stamina = max(
                        0.0,
                        self.stamina - 2.0
                    )
                    self.last_key = pygame.K_RIGHT

    def update_ai_state(self, dt):
        self.ai_state_timer += dt

        if self.ai_state == "NORMAL":
            if self.ai_state_timer >= self.ai_normal_duration:
                self.ai_state = "SURGE"
                self.ai_state_timer = 0.0

        elif self.ai_state == "SURGE":
            if self.ai_state_timer >= self.ai_surge_duration:
                self.ai_state = "COOLDOWN"
                self.ai_state_timer = 0.0

        elif self.ai_state == "COOLDOWN":
            if self.ai_state_timer >= self.ai_cooldown_duration:
```

```
            self.ai_state = "NORMAL"
            self.ai_state_timer = 0.0

    def get_ai_state_info(self):
        if self.ai_state == "SURGE":
            return (
                "POWER SURGE!",
                (255, 65, 65),
                self.ai_surge_duration
            )

        elif self.ai_state == "COOLDOWN":
            return (
                "AI COOLDOWN",
                (70, 230, 130),
                self.ai_cooldown_duration
            )

        return (
            "AI NORMAL",
            (220, 220, 220),
            self.ai_normal_duration
        )

    def update(self):
        if self.game_state != "PLAYING":
            return

        current_time = pygame.time.get_ticks()

        dt = (current_time - self.last_update_time) / 1000.0
        self.last_update_time = current_time

        # Avoid unusually large timer jumps
        dt = min(dt, 0.1)

        # Update AI cycle
        self.update_ai_state(dt)

        # Determine AI strength
        if self.ai_state == "SURGE":
```

```
            current_ai_strength = (
                self.ai_strength * self.ai_surge_multiplier
            )

        elif self.ai_state == "COOLDOWN":
            current_ai_strength = (
                self.ai_strength * self.ai_cooldown_multiplier
            )

        else:
            current_ai_strength = self.ai_strength

        # Apply opposing computer force
        ai_variance = random.uniform(0.3, 1.0)

        self.arm_position += (
            current_ai_strength * ai_variance
        )

        # Recover stamina
        if self.stamina < self.max_stamina:
            self.stamina = min(
                self.max_stamina,
                self.stamina + 0.8
            )

        # Check winning conditions
        if self.arm_position <= -self.target_limit:
            self.winner = "PLAYER"
            self.game_state = "GAME_OVER"

        elif self.arm_position >= self.target_limit:
            self.winner = "COMPUTER"
            self.game_state = "GAME_OVER"

    def reset(self):
        self.arm_position = 0.0
        self.stamina = 100.0
        self.last_key = None

        self.winner = None
```

```
        self.game_state = "PLAYING"

        self.ai_state = "NORMAL"
        self.ai_state_timer = 0.0

        self.last_update_time = pygame.time.get_ticks()

    def render(self, screen):
        screen.fill((25, 28, 35))

        # Title
        title_surf = self.font_big.render(
            "ARM WRESTLE SHOWDOWN",
            True,
            (240, 240, 240)
        )

        screen.blit(
            title_surf,
            (
                self.width // 2 - title_surf.get_width() // 2,
                12
            )
        )

        # Player and computer labels
        player_header = self.font_med.render(
            "PLAYER",
            True,
            (80, 160, 255)
        )

        computer_header = self.font_med.render(
            "COMPUTER",
            True,
            (255, 100, 80)
        )

        screen.blit(player_header, (60, 55))
        screen.blit(
            computer_header,
```

```
        (self.width - 150, 55)
    )

    # AI state indicator
    state_text, state_color, state_duration = (
        self.get_ai_state_info()
    )

    state_surface = self.font_med.render(
        state_text,
        True,
        state_color
    )

    screen.blit(
        state_surface,
        (
            self.width // 2 - state_surface.get_width() // 2,
            70
        )
    )

    # Countdown for current AI state
    remaining_time = max(
        0.0,
        state_duration - self.ai_state_timer
    )

    timer_text = self.font_small.render(
        f"Next change in: {remaining_time:.1f}s",
        True,
        (210, 210, 210)
    )

    screen.blit(
        timer_text,
        (
            self.width // 2 - timer_text.get_width() // 2,
            95
        )
    )
```

```
        # Table
        table_rect = pygame.Rect(
            40,
            130,
            self.width - 80,
            280
        )

        pygame.draw.rect(
            screen,
            (110, 50, 15),
            table_rect,
            border_radius=14
        )

        pygame.draw.rect(
            screen,
            (70, 30, 8),
            table_rect,
            width=5,
            border_radius=14
        )

        pygame.draw.line(
            screen,
            (45, 18, 4),
            (self.width // 2, 130),
            (self.width // 2, 410),
            4
        )

        # Arm position
        offset_x = (
            self.arm_position / self.target_limit
        ) * 95

        hand_x = (
            self.width // 2
        ) + int(offset_x)
```

```
        hand_y = 245

        p_shoulder = (70, 330)
        p_elbow = (140, 225)

        c_shoulder = (self.width - 70, 330)
        c_elbow = (self.width - 140, 225)

        # Player arm
        pygame.draw.line(
            screen,
            (200, 145, 110),
            p_shoulder,
            p_elbow,
            32
        )

        pygame.draw.line(
            screen,
            (215, 160, 125),
            p_elbow,
            (hand_x, hand_y),
            26
        )

        pygame.draw.circle(
            screen,
            (185, 130, 95),
            p_elbow,
            18
        )

        # Computer arm
        pygame.draw.line(
            screen,
            (170, 110, 85),
            c_shoulder,
            c_elbow,
            32
        )
```

```
        pygame.draw.line(
            screen,
            (185, 125, 95),
            c_elbow,
            (hand_x, hand_y),
            26
        )

        pygame.draw.circle(
            screen,
            (150, 95, 70),
            c_elbow,
            18
        )

        # Clasped hands
        pygame.draw.circle(
            screen,
            (225, 175, 140),
            (hand_x, hand_y),
            24
        )

        pygame.draw.circle(
            screen,
            (160, 115, 85),
            (hand_x, hand_y),
            24,
            width=3
        )

        # Stamina
        stamina_label = self.font_med.render(
            "STAMINA",
            True,
            (220, 220, 220)
        )

        screen.blit(stamina_label, (40, 445))

        stamina_bg = pygame.Rect(
```

```
            140,
            448,
            240,
            22
        )

        stamina_fill = pygame.Rect(
            140,
            448,
            int(
                240 * (
                    self.stamina / self.max_stamina
                )
            ),
            22
        )

        pygame.draw.rect(
            screen,
            (45, 50, 60),
            stamina_bg,
            border_radius=6
        )

        bar_color = (
            (60, 210, 100)
            if self.stamina > 25
            else (220, 60, 60)
        )

        pygame.draw.rect(
            screen,
            bar_color,
            stamina_fill,
            border_radius=6
        )

        # Game Over
        if self.game_state == "GAME_OVER":
            overlay = pygame.Surface(
                (self.width, self.height),
```

```
        pygame.SRCALPHA
    )

    overlay.fill((0, 0, 0, 200))
    screen.blit(overlay, (0, 0))

    win_text = (
        "PLAYER WINS THE MATCH!"
        if self.winner == "PLAYER"
        else "COMPUTER WINS!"
    )

    color = (
        (80, 240, 100)
        if self.winner == "PLAYER"
        else (240, 80, 80)
    )

    text_surf = self.font_big.render(
        win_text,
        True,
        color
    )

    screen.blit(
        text_surf,
        (
            self.width // 2 - text_surf.get_width() // 2,
            self.height // 2 - 45
        )
    )

    restart_surf = self.font_med.render(
        "Press [R] to Rematch",
        True,
        (240, 240, 240)
    )

    screen.blit(
        restart_surf,
        (
```

```
                self.width // 2 - restart_surf.get_width() // 2,
                self.height // 2 + 10
            )
        )
```

Step 3: Save and run

Save using **Cmd + S**, then run these commands from your `53_arm_wrestle` folder:

```
python3 -m py_compile game/game_engine.py
```

If that succeeds, run:

```
python3 main.py
```

Step 4: What you should see

The game now displays the AI's current state and a countdown.

State	Screen indicator	Duration	Force
Normal	AI NORMAL	5 seconds	1×
Surge	POWER SURGE!	2.5 seconds	2.5×
Cooldown	AI COOLDOWN	4 seconds	0.25×

The countdown resets whenever the AI changes state.

Test it for at least 12 seconds. You should see the indicator change from `AI NORMAL` to `POWER SURGE!`, then to `AI COOLDOWN`, and back to normal.

One limitation remains: this version displays a clear warning during the surge, but it doesn't yet provide a separate advance warning before the surge starts. We can add that when we implement the counter-surge feature.

Tell me whether you can now see the AI state and countdown changing.

You said:

yes i can see it change

ChatGPT said:

BROOO 🔥😭 NICE! The dynamic AI cycle is working!

We've completed the first two stages. Let's move to the next requirement.

✅ Progress so far

Requirement	Status
Fix inverted arm push	✅
Implement dynamic AI surge cycle	✅
Display AI state and countdown	✅
Preserve stamina and recovery	✅
Computer victory and restart	✅

Step 3: Implement the Counter-Surge Bonus ⚡

This is one of the more interesting features in your lab because it rewards the player for timing their inputs.

How it will work

When the AI's power surge ends and cooldown begins:

1. A **COUNTER-SURGE AVAILABLE!** indicator appears.
2. If you press Left or Right during the opening window, your push strength temporarily doubles from `4.2` to `8.4`.
3. Your stamina recovery temporarily increases.
4. The bonus expires after a short period, and normal gameplay resumes.

We'll use a **1.5-second activation window** and a **2-second bonus duration** to start with.

One important detail

Your current code already tracks the AI's state transitions. We'll build on that instead of introducing a separate timer system.

We'll also ensure the bonus can activate only once per AI surge-to-cooldown transition, preventing repeated presses from restarting it indefinitely.

Before we implement it

We'll modify `game/game_engine.py` again, but keep the existing state indicator, countdown, arm movement, and game-over screen.

Ready, bro? We'll add the counter-surge bonus next, then finish the low-stamina exhaustion indicator and run a final test of all four requirements.

You said:

yes

ChatGPT said:

BROOO 🔥 Let's add the **Counter-Surge Bonus** while keeping your working AI cycle, countdown, arm movement, and stamina system intact.

The new mechanic will reward you for reacting quickly when the computer's power surge ends.

Step 1: Replace `game/game_engine.py` completely

Open `game/game_engine.py` in VS Code, press **Cmd + A**, and replace the entire file with this code.

This version includes:

- ✅ Correct player push direction.
- ✅ Dynamic AI surge and cooldown cycle.
- ✅ Visible AI state and countdown.
- ✅ Counter-surge activation window.
- ✅ Double push strength during the bonus.
- ✅ Faster stamina recovery during the bonus.
- ✅ Stamina exhaustion indicator.
- ✅ Game Over and restart functionality.

```
import random
import pygame


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        # Arm position
        self.arm_position = 0.0
        self.target_limit = 100.0
```

```
        self.last_key = None

        # Player stamina
        self.stamina = 100.0
        self.max_stamina = 100.0

        # Game state
        self.winner = None
        self.game_state = "PLAYING"

        # AI configuration
        self.ai_strength = 0.35
        self.ai_state = "NORMAL"
        self.ai_state_timer = 0.0

        # AI cycle durations
        self.ai_normal_duration = 5.0
        self.ai_surge_duration = 2.5
        self.ai_cooldown_duration = 4.0

        # AI force multipliers
        self.ai_surge_multiplier = 2.5
        self.ai_cooldown_multiplier = 0.25

        # Counter-surge configuration
        self.counter_window_duration = 1.5
        self.counter_bonus_duration = 2.0
        self.counter_push_multiplier = 2.0

        self.counter_window_timer = 0.0
        self.counter_bonus_timer = 0.0

        self.counter_surge_available = False
        self.counter_surge_active = False
        self.counter_surge_used = False

        # Stamina configuration
        self.stamina_cost = 2.0
        self.stamina_recovery = 0.8
        self.counter_stamina_recovery = 2.0
```

```
        # Timing
        self.last_update_time = pygame.time.get_ticks()

        # Fonts
        self.font_big = pygame.font.SysFont(None, 44)
        self.font_med = pygame.font.SysFont(None, 26)
        self.font_small = pygame.font.SysFont(None, 22)

    def handle_event(self, event):
        # Restart after Game Over
        if self.game_state != "PLAYING":
            if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                self.reset()
            return

        if event.type != pygame.KEYDOWN:
            return

        # Ignore unrelated keys
        if event.key not in (pygame.K_LEFT, pygame.K_RIGHT):
            return

        # Block input when stamina is exhausted
        if self.stamina <= 10:
            return

        # Require alternating arrow keys
        if event.key == self.last_key:
            return

        # Determine push strength
        push_strength = 4.2

        if self.counter_surge_active:
            push_strength *= self.counter_push_multiplier

        # Push toward the player's winning threshold
        self.arm_position -= push_strength

        # Reduce stamina
        self.stamina = max(
```

```
            0.0,
            self.stamina - self.stamina_cost
        )

        self.last_key = event.key

        # Activate counter-surge if the player reacts in time
        if self.counter_surge_available:
            self.activate_counter_surge()

    def activate_counter_surge(self):
        if not self.counter_surge_available:
            return

        if self.counter_surge_used:
            return

        self.counter_surge_available = False
        self.counter_surge_used = True

        self.counter_surge_active = True
        self.counter_bonus_timer = self.counter_bonus_duration

    def update_ai_state(self, dt):
        previous_state = self.ai_state

        self.ai_state_timer += dt

        if self.ai_state == "NORMAL":
            if self.ai_state_timer >= self.ai_normal_duration:
                self.ai_state = "SURGE"
                self.ai_state_timer = 0.0

        elif self.ai_state == "SURGE":
            if self.ai_state_timer >= self.ai_surge_duration:
                self.ai_state = "COOLDOWN"
                self.ai_state_timer = 0.0

        elif self.ai_state == "COOLDOWN":
            if self.ai_state_timer >= self.ai_cooldown_duration:
                self.ai_state = "NORMAL"
```

```
                self.ai_state_timer = 0.0

        # Open the counter-surge window when a surge ends
        if previous_state == "SURGE" and self.ai_state == "COOLDOWN":
            self.counter_surge_available = True
            self.counter_window_timer = self.counter_window_duration

            self.counter_surge_used = False
            self.counter_surge_active = False
            self.counter_bonus_timer = 0.0

    def get_ai_state_info(self):
        if self.ai_state == "SURGE":
            return (
                "POWER SURGE!",
                (255, 65, 65),
                self.ai_surge_duration
            )

        if self.ai_state == "COOLDOWN":
            return (
                "AI COOLDOWN",
                (70, 230, 130),
                self.ai_cooldown_duration
            )

        return (
            "AI NORMAL",
            (220, 220, 220),
            self.ai_normal_duration
        )

    def update(self):
        if self.game_state != "PLAYING":
            return

        current_time = pygame.time.get_ticks()

        dt = (current_time - self.last_update_time) / 1000.0
        self.last_update_time = current_time
```

```
        # Prevent unusually large timer jumps
        dt = min(dt, 0.1)

        # Update AI state
        self.update_ai_state(dt)

        # Update counter-surge activation window
        if self.counter_surge_available:
            self.counter_window_timer -= dt

            if self.counter_window_timer <= 0:
                self.counter_window_timer = 0.0
                self.counter_surge_available = False

        # Update active counter-surge bonus
        if self.counter_surge_active:
            self.counter_bonus_timer -= dt

            if self.counter_bonus_timer <= 0:
                self.counter_bonus_timer = 0.0
                self.counter_surge_active = False

        # Determine AI force
        if self.ai_state == "SURGE":
            current_ai_strength = (
                self.ai_strength * self.ai_surge_multiplier
            )

        elif self.ai_state == "COOLDOWN":
            current_ai_strength = (
                self.ai_strength * self.ai_cooldown_multiplier
            )

        else:
            current_ai_strength = self.ai_strength

        # Apply opposing computer force
        ai_variance = random.uniform(0.3, 1.0)

        self.arm_position += current_ai_strength * ai_variance
```

```
        # Recover stamina
        if self.stamina < self.max_stamina:
            if self.counter_surge_active:
                recovery = self.counter_stamina_recovery
            else:
                recovery = self.stamina_recovery

            self.stamina = min(
                self.max_stamina,
                self.stamina + recovery
            )

        # Check winning conditions
        if self.arm_position <= -self.target_limit:
            self.winner = "PLAYER"
            self.game_state = "GAME_OVER"

        elif self.arm_position >= self.target_limit:
            self.winner = "COMPUTER"
            self.game_state = "GAME_OVER"

    def reset(self):
        # Reset arm and stamina
        self.arm_position = 0.0
        self.stamina = 100.0
        self.last_key = None

        # Reset game state
        self.winner = None
        self.game_state = "PLAYING"

        # Reset AI cycle
        self.ai_state = "NORMAL"
        self.ai_state_timer = 0.0

        # Reset counter-surge
        self.counter_window_timer = 0.0
        self.counter_bonus_timer = 0.0

        self.counter_surge_available = False
        self.counter_surge_active = False
```

```
        self.counter_surge_used = False

        # Reset timing
        self.last_update_time = pygame.time.get_ticks()

    def render(self, screen):
        screen.fill((25, 28, 35))

        # Title
        title_surf = self.font_big.render(
            "ARM WRESTLE SHOWDOWN",
            True,
            (240, 240, 240)
        )

        screen.blit(
            title_surf,
            (
                self.width // 2 - title_surf.get_width() // 2,
                12
            )
        )

        # Player and computer labels
        player_header = self.font_med.render(
            "PLAYER",
            True,
            (80, 160, 255)
        )

        computer_header = self.font_med.render(
            "COMPUTER",
            True,
            (255, 100, 80)
        )

        screen.blit(player_header, (60, 55))
        screen.blit(
            computer_header,
            (self.width - 150, 55)
        )
```

```
        # AI state indicator
        state_text, state_color, state_duration = (
            self.get_ai_state_info()
        )

        state_surface = self.font_med.render(
            state_text,
            True,
            state_color
        )

        screen.blit(
            state_surface,
            (
                self.width // 2 - state_surface.get_width() // 2,
                70
            )
        )

        # AI state countdown
        remaining_time = max(
            0.0,
            state_duration - self.ai_state_timer
        )

        timer_text = self.font_small.render(
            f"Next change in: {remaining_time:.1f}s",
            True,
            (210, 210, 210)
        )

        screen.blit(
            timer_text,
            (
                self.width // 2 - timer_text.get_width() // 2,
                95
            )
        )

        # Counter-surge warning
```

```
        if self.counter_surge_available:
            warning_text = (
                f"COUNTER-SURGE AVAILABLE! "
                f"{self.counter_window_timer:.1f}s"
            )

            warning_surface = self.font_small.render(
                warning_text,
                True,
                (255, 220, 60)
            )

            screen.blit(
                warning_surface,
                (
                    self.width // 2 - warning_surface.get_width() // 2,
                    118
                )
            )

        elif self.counter_surge_active:
            bonus_text = (
                f"COUNTER-SURGE ACTIVE! "
                f"{self.counter_bonus_timer:.1f}s"
            )

            bonus_surface = self.font_small.render(
                bonus_text,
                True,
                (80, 255, 150)
            )

            screen.blit(
                bonus_surface,
                (
                    self.width // 2 - bonus_surface.get_width() // 2,
                    118
                )
            )

        # Table
```

```
        table_rect = pygame.Rect(
            40,
            145,
            self.width - 80,
            265
        )

        pygame.draw.rect(
            screen,
            (110, 50, 15),
            table_rect,
            border_radius=14
        )

        pygame.draw.rect(
            screen,
            (70, 30, 8),
            table_rect,
            width=5,
            border_radius=14
        )

        pygame.draw.line(
            screen,
            (45, 18, 4),
            (self.width // 2, 145),
            (self.width // 2, 410),
            4
        )

        # Arm position
        offset_x = (
            self.arm_position / self.target_limit
        ) * 95

        hand_x = (
            self.width // 2
        ) + int(offset_x)

        hand_y = 250
```

```
        p_shoulder = (70, 330)
        p_elbow = (140, 225)

        c_shoulder = (self.width - 70, 330)
        c_elbow = (self.width - 140, 225)

        # Player arm
        pygame.draw.line(
            screen,
            (200, 145, 110),
            p_shoulder,
            p_elbow,
            32
        )

        pygame.draw.line(
            screen,
            (215, 160, 125),
            p_elbow,
            (hand_x, hand_y),
            26
        )

        pygame.draw.circle(
            screen,
            (185, 130, 95),
            p_elbow,
            18
        )

        # Computer arm
        pygame.draw.line(
            screen,
            (170, 110, 85),
            c_shoulder,
            c_elbow,
            32
        )

        pygame.draw.line(
            screen,
```

```
            (185, 125, 95),
            c_elbow,
            (hand_x, hand_y),
            26
        )

        pygame.draw.circle(
            screen,
            (150, 95, 70),
            c_elbow,
            18
        )

        # Clasped hands
        pygame.draw.circle(
            screen,
            (225, 175, 140),
            (hand_x, hand_y),
            24
        )

        pygame.draw.circle(
            screen,
            (160, 115, 85),
            (hand_x, hand_y),
            24,
            width=3
        )

        # Stamina label
        stamina_label = self.font_med.render(
            "STAMINA",
            True,
            (220, 220, 220)
        )

        screen.blit(stamina_label, (40, 445))

        # Stamina bar
        stamina_bg = pygame.Rect(
            140,
```

```
            448,
            240,
            22
        )

        stamina_fill = pygame.Rect(
            140,
            448,
            int(
                240 * (
                    self.stamina / self.max_stamina
                )
            ),
            22
        )

        pygame.draw.rect(
            screen,
            (45, 50, 60),
            stamina_bg,
            border_radius=6
        )

        bar_color = (
            (60, 210, 100)
            if self.stamina > 25
            else (220, 60, 60)
        )

        pygame.draw.rect(
            screen,
            bar_color,
            stamina_fill,
            border_radius=6
        )

        # Exhaustion warning
        if self.stamina <= 10:
            exhausted_surface = self.font_med.render(
                "EXHAUSTED! RECOVERING...",
                True,
```

```
            (255, 65, 65)
        )

        screen.blit(
            exhausted_surface,
            (
                self.width // 2
                - exhausted_surface.get_width() // 2,
                485
            )
        )

    # Game Over screen
    if self.game_state == "GAME_OVER":
        overlay = pygame.Surface(
            (self.width, self.height),
            pygame.SRCALPHA
        )

        overlay.fill((0, 0, 0, 200))
        screen.blit(overlay, (0, 0))

        win_text = (
            "PLAYER WINS THE MATCH!"
            if self.winner == "PLAYER"
            else "COMPUTER WINS!"
        )

        color = (
            (80, 240, 100)
            if self.winner == "PLAYER"
            else (240, 80, 80)
        )

        text_surf = self.font_big.render(
            win_text,
            True,
            color
        )

        screen.blit(
```

```
            text_surf,
            (
                self.width // 2 - text_surf.get_width() // 2,
                self.height // 2 - 45
            )
        )

        restart_surf = self.font_med.render(
            "Press [R] to Rematch",
            True,
            (240, 240, 240)
        )

        screen.blit(
            restart_surf,
            (
                self.width // 2 - restart_surf.get_width() // 2,
                self.height // 2 + 10
            )
        )
```

Step 2: Save and run the game

Save the file with **Cmd + S**.

In your terminal, run:

```
python3 -m py_compile game/game_engine.py
```

If there are no errors, run:

```
python3 main.py
```

You don't need to modify `main.py`.

Step 3: Test the counter-surge bonus

Play for at least 8 seconds until the AI finishes its first surge and enters cooldown.

You'll see:

`COUNTER-SURGE AVAILABLE! 1.5s`

Immediately press an arrow key that differs from your previous accepted key.

If you activate the bonus successfully, you'll see:

`COUNTER-SURGE ACTIVE! 2.0s`

During the bonus:

- Your push strength doubles from `4.2` to `8.4`.
- Stamina recovery increases from `0.8` to `2.0` per update.
- The bonus lasts for 2 seconds.

If you miss the activation window, don't worry. You can try again after the next AI surge ends.

One detail worth knowing

Your stamina currently recovers every game update, not every second. At 60 FPS, recovery of `0.8` per update is quite fast. I've preserved that existing behaviour to avoid changing the balance of the game unexpectedly.

Run it and tell me whether you see the counter-surge warning and can activate the bonus. We'll verify this before making any further changes.

You said:

i dont see any counter surge

ChatGPT said:

Bro, I found the likely problem. 😭 The counter-surge logic is there, but **the computer can win before the first counter-surge window even opens.**

Your AI waits 5 seconds in NORMAL, then spends 2.5 seconds in SURGE. That's 7.5 seconds before the counter-surge becomes available. Meanwhile, the computer's force is applied on every frame, so the arm can reach the winning threshold very quickly.

Let's fix the underlying timing rather than just changing the warning.

Step 1: Make these changes in `game/game_engine.py`

Change 1: Adjust the AI strength

Find this line in `__init__`:

```
self.ai_strength = 0.35
```

Replace it with:

```
self.ai_strength = 8.0
```

We'll now apply the AI force using elapsed time, so this represents force per second rather than force per frame.

Change 2: Make the AI force depend on elapsed time

Inside `update()`, find this code:

```
ai_variance = random.uniform(0.3, 1.0)

self.arm_position += current_ai_strength * ai_variance
```

Replace it with:

```
ai_variance = random.uniform(0.3, 1.0)

# Apply AI force based on elapsed time.
# This prevents the AI from pushing excessively fast at 60 FPS.
self.arm_position += current_ai_strength * ai_variance * dt
```

Change 3: Make sure the counter-surge window opens

In `update_ai_state()`, find this condition:

```
if previous_state == "SURGE" and self.ai_state == "COOLDOWN":
```

Replace that entire condition and its body with:

```
if previous_state == "SURGE" and self.ai_state == "COOLDOWN":
    self.counter_surge_available = True
    self.counter_window_timer = self.counter_window_duration

    self.counter_surge_used = False
    self.counter_surge_active = False
    self.counter_bonus_timer = 0.0

    print("COUNTER-SURGE WINDOW OPEN!")
```

The terminal message will help us confirm that the transition actually happens.

Step 2: Save and run

Save using **Cmd + S**, then run:

```
python3 -m py_compile game/game_engine.py
```

If there are no errors:

```
python3 main.py
```

Wait for the AI to complete its first surge.

You should see the counter-surge warning, and your terminal should print:

```
COUNTER-SURGE WINDOW OPEN!
```

When the warning appears, press Left or Right, making sure you alternate from your previous accepted key.

Important: These are three targeted changes, not a complete replacement of your file. Leave the rest of your code unchanged for now.

Tell me whether you see the warning and the terminal message. Then we'll verify that the doubled push strength and faster stamina recovery actually work.

You said:

give me final file

ChatGPT said:

Bro, got you. 💪 We're fixing the counter-surge properly this time.

This complete file includes:

- ✅ Correct player push direction.
- ✅ Dynamic AI normal, surge, and cooldown cycle.
- ✅ Visible AI state and countdown.
- ✅ Counter-surge warning when cooldown begins.
- ✅ Double player push strength during the bonus.
- ✅ Faster stamina recovery during the bonus.
- ✅ Exhaustion indicator.
- ✅ Player and computer win conditions.
- ✅ Restart using `R`.

I've also corrected the AI force to be **time-based**, so its movement doesn't become excessively fast at 60 FPS.

Step 1: Replace `game/game_engine.py` completely

Open the file, press **Cmd + A**, and replace everything with this code.

```
import random
import pygame


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        # Arm position
        self.arm_position = 0.0
        self.target_limit = 100.0
        self.last_key = None

        # Player stamina
        self.stamina = 100.0
        self.max_stamina = 100.0
        self.stamina_cost = 2.0
        self.stamina_recovery = 12.0
        self.counter_stamina_recovery = 30.0

        # Game state
        self.winner = None
        self.game_state = "PLAYING"

        # AI configuration: force units per second
        self.ai_strength = 8.0
        self.ai_state = "NORMAL"
        self.ai_state_timer = 0.0

        # AI cycle durations in seconds
        self.ai_normal_duration = 5.0
        self.ai_surge_duration = 2.5
        self.ai_cooldown_duration = 4.0

        # AI force multipliers
        self.ai_surge_multiplier = 2.5
```

```
        self.ai_cooldown_multiplier = 0.25

        # Counter-surge configuration
        self.counter_window_duration = 1.5
        self.counter_bonus_duration = 2.0
        self.counter_push_multiplier = 2.0

        self.counter_window_timer = 0.0
        self.counter_bonus_timer = 0.0

        self.counter_surge_available = False
        self.counter_surge_active = False
        self.counter_surge_used = False

        # Timing
        self.last_update_time = pygame.time.get_ticks()

        # Fonts
        self.font_big = pygame.font.SysFont(None, 44)
        self.font_med = pygame.font.SysFont(None, 26)
        self.font_small = pygame.font.SysFont(None, 22)

    def handle_event(self, event):
        # Restart after Game Over
        if self.game_state != "PLAYING":
            if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                self.reset()
            return

        if event.type != pygame.KEYDOWN:
            return

        # Only arrow keys affect the arm
        if event.key not in (pygame.K_LEFT, pygame.K_RIGHT):
            return

        # Block input when stamina is exhausted
        if self.stamina <= 10:
            return

        # Require alternating arrow keys
```

```
        if event.key == self.last_key:
            return

        # Determine player push strength
        push_strength = 4.2

        if self.counter_surge_active:
            push_strength *= self.counter_push_multiplier

        # Push toward the player's winning threshold
        self.arm_position -= push_strength

        # Reduce stamina
        self.stamina = max(
            0.0,
            self.stamina - self.stamina_cost
        )

        self.last_key = event.key

        # Activate the bonus if the player reacts in time
        if self.counter_surge_available:
            self.activate_counter_surge()

    def activate_counter_surge(self):
        if not self.counter_surge_available:
            return

        if self.counter_surge_used:
            return

        self.counter_surge_available = False
        self.counter_surge_used = True
        self.counter_surge_active = True
        self.counter_bonus_timer = self.counter_bonus_duration

        print("COUNTER-SURGE ACTIVATED!")

    def update_ai_state(self, dt):
        previous_state = self.ai_state
```

```
        self.ai_state_timer += dt

        if self.ai_state == "NORMAL":
            if self.ai_state_timer >= self.ai_normal_duration:
                self.ai_state = "SURGE"
                self.ai_state_timer = 0.0
                print("AI POWER SURGE!")

        elif self.ai_state == "SURGE":
            if self.ai_state_timer >= self.ai_surge_duration:
                self.ai_state = "COOLDOWN"
                self.ai_state_timer = 0.0
                print("AI ENTERING COOLDOWN!")

        elif self.ai_state == "COOLDOWN":
            if self.ai_state_timer >= self.ai_cooldown_duration:
                self.ai_state = "NORMAL"
                self.ai_state_timer = 0.0
                print("AI RETURNING TO NORMAL!")

        # Open the counter-surge window when the surge ends
        if (
            previous_state == "SURGE"
            and self.ai_state == "COOLDOWN"
        ):
            self.counter_surge_available = True
            self.counter_window_timer = self.counter_window_duration

            self.counter_surge_used = False
            self.counter_surge_active = False
            self.counter_bonus_timer = 0.0

            print("COUNTER-SURGE WINDOW OPEN!")

    def get_ai_state_info(self):
        if self.ai_state == "SURGE":
            return (
                "POWER SURGE!",
                (255, 65, 65),
                self.ai_surge_duration
            )
```

```
        if self.ai_state == "COOLDOWN":
            return (
                "AI COOLDOWN",
                (70, 230, 130),
                self.ai_cooldown_duration
            )

        return (
            "AI NORMAL",
            (220, 220, 220),
            self.ai_normal_duration
        )

    def update(self):
        if self.game_state != "PLAYING":
            return

        current_time = pygame.time.get_ticks()

        dt = (
            current_time - self.last_update_time
        ) / 1000.0

        self.last_update_time = current_time

        # Avoid huge timer jumps after pauses
        dt = min(dt, 0.1)

        # Update AI state
        self.update_ai_state(dt)

        # Update counter-surge activation window
        if self.counter_surge_available:
            self.counter_window_timer -= dt

            if self.counter_window_timer <= 0:
                self.counter_window_timer = 0.0
                self.counter_surge_available = False
                print("COUNTER-SURGE WINDOW EXPIRED!")
```

```
        # Update active bonus
        if self.counter_surge_active:
            self.counter_bonus_timer -= dt

            if self.counter_bonus_timer <= 0:
                self.counter_bonus_timer = 0.0
                self.counter_surge_active = False
                print("COUNTER-SURGE BONUS ENDED!")

        # Determine AI force
        if self.ai_state == "SURGE":
            current_ai_strength = (
                self.ai_strength * self.ai_surge_multiplier
            )

        elif self.ai_state == "COOLDOWN":
            current_ai_strength = (
                self.ai_strength * self.ai_cooldown_multiplier
            )

        else:
            current_ai_strength = self.ai_strength

        # Apply time-based opposing computer force
        ai_variance = random.uniform(0.3, 1.0)

        self.arm_position += (
            current_ai_strength * ai_variance * dt
        )

        # Recover stamina
        if self.stamina < self.max_stamina:
            if self.counter_surge_active:
                recovery = self.counter_stamina_recovery
            else:
                recovery = self.stamina_recovery

            self.stamina = min(
                self.max_stamina,
                self.stamina + recovery * dt
            )
```

```
        # Check winning conditions
        if self.arm_position <= -self.target_limit:
            self.winner = "PLAYER"
            self.game_state = "GAME_OVER"

        elif self.arm_position >= self.target_limit:
            self.winner = "COMPUTER"
            self.game_state = "GAME_OVER"

    def reset(self):
        # Reset arm and stamina
        self.arm_position = 0.0
        self.stamina = 100.0
        self.last_key = None

        # Reset game state
        self.winner = None
        self.game_state = "PLAYING"

        # Reset AI cycle
        self.ai_state = "NORMAL"
        self.ai_state_timer = 0.0

        # Reset counter-surge
        self.counter_window_timer = 0.0
        self.counter_bonus_timer = 0.0

        self.counter_surge_available = False
        self.counter_surge_active = False
        self.counter_surge_used = False

        # Reset timing
        self.last_update_time = pygame.time.get_ticks()

    def render(self, screen):
        screen.fill((25, 28, 35))

        # Title
        title_surf = self.font_big.render(
            "ARM WRESTLE SHOWDOWN",
```

```
            True,
            (240, 240, 240)
        )

        screen.blit(
            title_surf,
            (
                self.width // 2 - title_surf.get_width() // 2,
                12
            )
        )

        # Player and computer labels
        player_header = self.font_med.render(
            "PLAYER",
            True,
            (80, 160, 255)
        )

        computer_header = self.font_med.render(
            "COMPUTER",
            True,
            (255, 100, 80)
        )

        screen.blit(player_header, (60, 55))
        screen.blit(
            computer_header,
            (self.width - 150, 55)
        )

        # AI state indicator
        state_text, state_color, state_duration = (
            self.get_ai_state_info()
        )

        state_surface = self.font_med.render(
            state_text,
            True,
            state_color
        )
```

```
screen.blit(
    state_surface,
    (
        self.width // 2 - state_surface.get_width() // 2,
        70
    )
)

# AI countdown
remaining_time = max(
    0.0,
    state_duration - self.ai_state_timer
)

timer_text = self.font_small.render(
    f"Next change in: {remaining_time:.1f}s",
    True,
    (210, 210, 210)
)

screen.blit(
    timer_text,
    (
        self.width // 2 - timer_text.get_width() // 2,
        95
    )
)

# Counter-surge warning
if self.counter_surge_available:
    warning_text = (
        f"COUNTER-SURGE AVAILABLE! "
        f"{self.counter_window_timer:.1f}s"
    )

    warning_surface = self.font_small.render(
        warning_text,
        True,
        (255, 220, 60)
    )
```

```
            screen.blit(
                warning_surface,
                (
                    self.width // 2 - warning_surface.get_width() // 2,
                    118
                )
            )

        elif self.counter_surge_active:
            bonus_text = (
                f"COUNTER-SURGE ACTIVE! "
                f"{self.counter_bonus_timer:.1f}s"
            )

            bonus_surface = self.font_small.render(
                bonus_text,
                True,
                (80, 255, 150)
            )

            screen.blit(
                bonus_surface,
                (
                    self.width // 2 - bonus_surface.get_width() // 2,
                    118
                )
            )

        # Table
        table_rect = pygame.Rect(
            40,
            145,
            self.width - 80,
            265
        )

        pygame.draw.rect(
            screen,
            (110, 50, 15),
            table_rect,
```

```
        border_radius=14
    )

    pygame.draw.rect(
        screen,
        (70, 30, 8),
        table_rect,
        width=5,
        border_radius=14
    )

    pygame.draw.line(
        screen,
        (45, 18, 4),
        (self.width // 2, 145),
        (self.width // 2, 410),
        4
    )

    # Arm position
    offset_x = (
        self.arm_position / self.target_limit
    ) * 95

    hand_x = (
        self.width // 2
    ) + int(offset_x)

    hand_y = 250

    p_shoulder = (70, 330)
    p_elbow = (140, 225)

    c_shoulder = (self.width - 70, 330)
    c_elbow = (self.width - 140, 225)

    # Player arm
    pygame.draw.line(
        screen,
        (200, 145, 110),
        p_shoulder,
```

```
        p_elbow,
        32
    )

    pygame.draw.line(
        screen,
        (215, 160, 125),
        p_elbow,
        (hand_x, hand_y),
        26
    )

    pygame.draw.circle(
        screen,
        (185, 130, 95),
        p_elbow,
        18
    )

    # Computer arm
    pygame.draw.line(
        screen,
        (170, 110, 85),
        c_shoulder,
        c_elbow,
        32
    )

    pygame.draw.line(
        screen,
        (185, 125, 95),
        c_elbow,
        (hand_x, hand_y),
        26
    )

    pygame.draw.circle(
        screen,
        (150, 95, 70),
        c_elbow,
        18
```

```
        )

        # Clasped hands
        pygame.draw.circle(
            screen,
            (225, 175, 140),
            (hand_x, hand_y),
            24
        )

        pygame.draw.circle(
            screen,
            (160, 115, 85),
            (hand_x, hand_y),
            24,
            width=3
        )

        # Stamina
        stamina_label = self.font_med.render(
            "STAMINA",
            True,
            (220, 220, 220)
        )

        screen.blit(stamina_label, (40, 445))

        stamina_bg = pygame.Rect(
            140,
            448,
            240,
            22
        )

        stamina_fill = pygame.Rect(
            140,
            448,
            int(
                240 * (
                    self.stamina / self.max_stamina
                )
```

```
        ),
        22
    )

    pygame.draw.rect(
        screen,
        (45, 50, 60),
        stamina_bg,
        border_radius=6
    )

    bar_color = (
        (60, 210, 100)
        if self.stamina > 25
        else (220, 60, 60)
    )

    pygame.draw.rect(
        screen,
        bar_color,
        stamina_fill,
        border_radius=6
    )

    # Exhaustion warning
    if self.stamina <= 10:
        exhausted_surface = self.font_med.render(
            "EXHAUSTED! RECOVERING...",
            True,
            (255, 65, 65)
        )

        screen.blit(
            exhausted_surface,
            (
                self.width // 2
                - exhausted_surface.get_width() // 2,
                485
            )
        )
```

```
        # Game Over screen
        if self.game_state == "GAME_OVER":
            overlay = pygame.Surface(
                (self.width, self.height),
                pygame.SRCALPHA
            )

            overlay.fill((0, 0, 0, 200))
            screen.blit(overlay, (0, 0))

            win_text = (
                "PLAYER WINS THE MATCH!"
                if self.winner == "PLAYER"
                else "COMPUTER WINS!"
            )

            color = (
                (80, 240, 100)
                if self.winner == "PLAYER"
                else (240, 80, 80)
            )

            text_surf = self.font_big.render(
                win_text,
                True,
                color
            )

            screen.blit(
                text_surf,
                (
                    self.width // 2 - text_surf.get_width() // 2,
                    self.height // 2 - 45
                )
            )

            restart_surf = self.font_med.render(
                "Press [R] to Rematch",
                True,
                (240, 240, 240)
            )
```

```
            screen.blit(
                restart_surf,
                (
                    self.width // 2 - restart_surf.get_width() // 2,
                    self.height // 2 + 10
                )
            )
```

Step 2: Save and run

Save using **Cmd + S**.

In your terminal, run:

```
python3 -m py_compile game/game_engine.py
```

If that succeeds, run:

```
python3 main.py
```

Step 3: Test the counter-surge

Wait for the first AI surge to finish. The sequence should be:

1. **AI NORMAL** — 5 seconds.
2. **POWER SURGE!** — 2.5 seconds.
3. **AI COOLDOWN** — the counter-surge window opens for 1.5 seconds.

When you see `COUNTER-SURGE AVAILABLE!`, immediately press an arrow key different from your previous accepted key.

If successful, the screen displays `COUNTER-SURGE ACTIVE!` for 2 seconds.

You can also check your terminal for:

```
AI POWER SURGE!
AI ENTERING COOLDOWN!
COUNTER-SURGE WINDOW OPEN!
COUNTER-SURGE ACTIVATED!
```

One important distinction: The counter-surge doubles the player's push strength and increases stamina recovery, but it does not automatically win the match. You still need to push the arm to `-100`.

Tell me whether the warning appears and whether you can activate the bonus.

You said:

yes i can

ChatGPT said:

BROOOO 😭🔥 WE GOT IT WORKING!!

Your counter-surge bonus is now working, and the core gameplay features are implemented.

✅ Arm Wrestle Showdown — Progress Report

Requirement	Status
1. Fix inverted arm push	✅ Working
2. Dynamic AI surge and cooldown	✅ Working
3. Visual AI surge warning	✅ Implemented
4. Player exhaustion indicator	✅ Implemented
5. Counter-surge bonus	✅ Working
Double push strength during counter-surge	✅ Implemented
Increased stamina recovery during bonus	✅ Implemented
Player and computer win conditions	✅ Working
Restart using R	✅ Working

Before we call the project finished

I recommend one final round of testing. We want to make sure the features work together, not just individually.

Run these four tests:

1. **Exhaustion:** Alternate the arrow keys rapidly and check whether the exhaustion warning appears when stamina falls to 10 or below.
2. **Recovery:** Stop pressing keys and verify that stamina recovers and player input becomes available again.

3. **Counter-surge expiration:** Allow the 1.5-second activation window to expire without pressing an arrow key. Confirm that the bonus doesn't activate.
4. **Complete reset:** Finish a match, press `R`, and verify that the arm, stamina, AI cycle, and counter-surge timers reset.

One technical detail worth checking

We changed the AI force to be time-based, which makes its behaviour more consistent across different frame rates. That's a good improvement for your project.

However, your player push is still applied per key press, while stamina recovery is time-based. That's reasonable for this game, but we should test the stamina behaviour at the exhaustion threshold.

Bro, do those four tests and tell me if they pass. If they do, we'll have a much stronger basis for calling the implementation complete and preparing your lab demonstration.

You said:

```
Hello from the pygame community.
https://www.pygame.org/contribute.html
AI POWER SURGE!
AI ENTERING COOLDOWN!
COUNTER-SURGE WINDOW OPEN!
COUNTER-SURGE WINDOW EXPIRED!
AI RETURNING TO NORMAL!
AI POWER SURGE!
AI ENTERING COOLDOWN!
COUNTER-SURGE WINDOW OPEN!
COUNTER-SURGE ACTIVATED!
COUNTER-SURGE BONUS ENDED!
AI RETURNING TO NORMAL!
AI POWER SURGE!
AI ENTERING COOLDOWN!
COUNTER-SURGE WINDOW OPEN!
COUNTER-SURGE ACTIVATED!
COUNTER-SURGE BONUS ENDED!
AI RETURNING TO NORMAL!
```

ChatGPT said:

Bro, these logs look **good!** Your AI cycle and counter-surge mechanics are running correctly. ✅

Here's what your output confirms:

- **AI POWER SURGE!** — The AI enters its stronger attack phase.
- **AI ENTERING COOLDOWN!** — The AI transitions into its weaker phase.
- **COUNTER-SURGE WINDOW OPEN!** — You get a limited opportunity to counter.
- **COUNTER-SURGE WINDOW EXPIRED!** — The opportunity closes if you don't activate it in time.
- **COUNTER-SURGE ACTIVATED!** — Your counter-surge successfully activates.
- **COUNTER-SURGE BONUS ENDED!** — The temporary bonus ends.
- **AI RETURNING TO NORMAL!** — The AI cycle restarts.

The fact that the counter-surge activates in some cycles but expires in another is exactly what we'd expect.

Now test these 3 things

1. Stamina exhaustion

- Rapidly alternate Left and Right.
- Check whether stamina drops and the exhaustion warning appears at 10 or below.
- Stop pressing and confirm stamina recovers.

2. Counter-surge effect

- Wait for `COUNTER-SURGE WINDOW OPEN!`.
- Immediately alternate the arrow keys.
- Check whether your arm moves toward the player's side more effectively during the bonus.

3. Complete reset

- Finish a match.
- Press `R`.
- Confirm the arm returns to the center, stamina returns to 100, and the AI cycle restarts.

Tell me what happens with the stamina test first, bro. That's the next thing we need to validate before moving on to the final submission.

You said:

yes i tested it
it worked

ChatGPT said:

BROOO 😭🔥 okay, your core game mechanics are working!

✅ Arm Wrestle Showdown — Testing Status

- ✅ Game launches and renders correctly.
- ✅ Player controls work with alternating arrow keys.
- ✅ AI power surge, cooldown, and normal phases work.
- ✅ Counter-surge window opens at the right time.
- ✅ Counter-surge activates and the bonus expires correctly.
- ✅ Stamina decreases, exhaustion activates, and stamina recovers.
- ✅ Game resets correctly after a match.

Next: Let's finish Lab 4 submission

Before we submit, we should verify the **assignment requirements** so we don't miss any deliverables.

1. **Code:** `main.py` and `game/game_engine.py`
2. **README:** Project description, controls, features, and instructions to run the game.
3. **GitHub:** Make sure all files are committed and pushed.
4. **Screenshots:** Capture the running game, AI power surge, counter-surge activation, stamina exhaustion, and game-over screen if screenshots are required.

Bro, send me the Lab 4 submission instructions or rubric. I'll compare them against what we've built and tell you exactly what's left, without making you redo working code.

You said:

Upload the following to your Lab repo under Lab-4 folder,
a. Videos (before and after)
b. Updated Code

ChatGPT said:

Bro, the submission is actually pretty straightforward! 😭 We need to upload **two things inside your existing Lab repo's `Lab-4` folder**.

1. What you need to submit

Your folder should look like this:

```
Your-Lab-Repo/
└── Lab-4/
    ├── before.mp4
    ├── after.mp4
    ├── main.py
    ├── README.md
    └── game/
        └── game_engine.py
```

- `before.mp4` — Video of the original game before your modifications.
- `after.mp4` — Video of the updated Arm Wrestle Showdown with your new features.
- **Updated code** — Your final `main.py`, `game/game_engine.py`, and any other project files required to run the game.

Keep the existing folder and file names if your lab repo already has a specific structure.

2. Record the videos

Before video

Show the original game before your changes.

If you no longer have the original version, check your GitHub repository's commit history to retrieve it. Don't recreate the original from memory.

After video

Record your updated game demonstrating:

1. The game launching.
2. Player controls using alternating Left and Right arrow keys.
3. The AI power surge and cooldown indicators.
4. The counter-surge window and activation.
5. Stamina depletion and recovery.
6. The game-over screen and restart using `R`.

Keep the video clear and concise. Make sure the new features are visible rather than just showing the game running.

On your Mac, you can use **Shift + Command + 5** to open the screen-recording controls.

3. Upload the updated code

Your final folder should contain the complete working project:

```
Lab-4/
├── main.py
├── README.md
└── game/
    └── game_engine.py
```

Include any additional assets or files your game needs to run.

4. Push everything to GitHub Desktop

Since you're using GitHub Desktop:

1. Copy the videos and updated code into your repository's `Lab-4` folder.
2. Open GitHub Desktop and select your lab repository.
3. Check the **Changes** tab to ensure the new files appear.
4. Enter a commit message, such as `Complete Lab 4 Arm Wrestle Showdown .`
5. Click **Commit to main** (or your current branch).
6. Click **Push origin** to upload everything to GitHub.

Important

Videos can be large. If GitHub rejects them because they exceed its file-size limit, tell me the error rather than deleting or compressing files blindly.

One thing before we proceed, bro.

Do you already have a video of the original game before our modifications, or do you only have the updated game right now?

That determines our next step.

You said:

how to download chatgpt chatr as pdf

ChatGPT said:

Bro, you can save your ChatGPT conversation as a PDF on your Mac in a few steps! 😭

✅ Method 1: Save this chat as a PDF (easiest)

1. **Open the ChatGPT conversation** you want to save.
2. Press `⌘ Command + P` on your Mac.
3. The print window should open.
4. Look for the **PDF** dropdown at the bottom of the window.
5. Click it and select **Save as PDF**.
6. Choose where to save the file, give it a name, and click **Save**.

Done! Your conversation is now saved as a PDF.

⚠️ If Command + P doesn't work

Try opening the conversation in your web browser:

1. Open ChatGPT in Safari or Chrome.
2. Navigate to the conversation you want.
3. Press `⌘ + P`.
4. Select **Save as PDF**.

Method 2: Export your ChatGPT data

If you want to download your chat history rather than just one conversation:

1. Open ChatGPT.
2. Go to **Settings**.
3. Open **Data Controls**.
4. Select **Export Data**.
5. Confirm the export request.
6. Follow the instructions in the email you receive to download your data.

This exports your ChatGPT data; it isn't a direct PDF export of an individual conversation.

My recommendation

For your Lab 4 documentation, use **Method 1** to save the relevant conversation as a PDF.

If you want to submit the chat as evidence, open the PDF afterward and check that the important messages and code are visible before submitting.